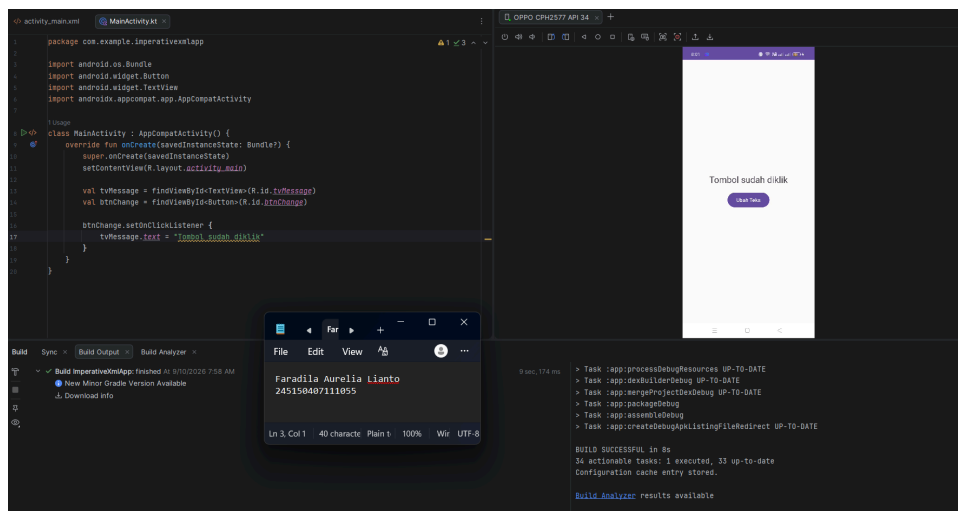


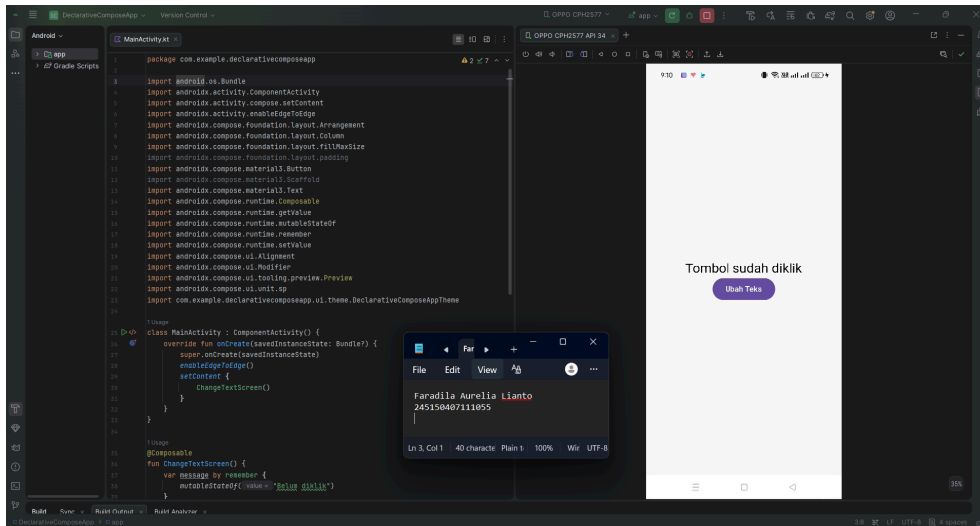
Nama : Faradila Aurelia Lianto
NIM : 245150407111055
Kelas : PAM-F

Modul 3: Event, State, dan Implisit Intent pada Jetpack Compose

Project A - XML/View (Imperatif)



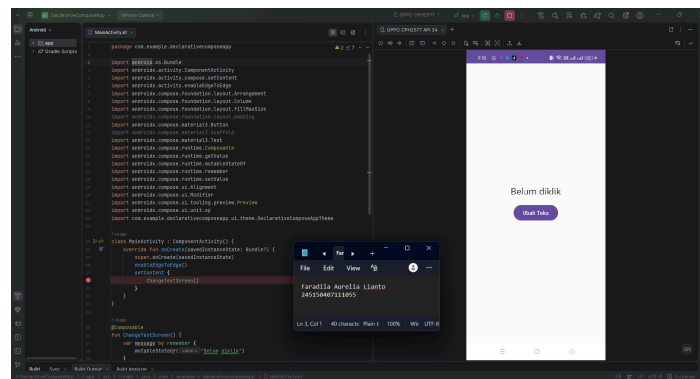
Project B - Jetpack Compose (Deklaratif)



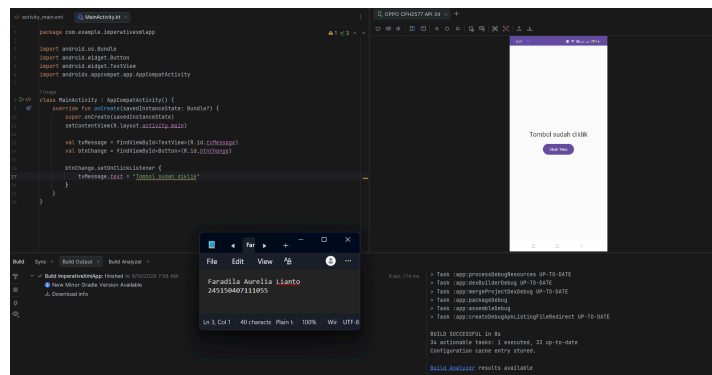
Aktivitas Percobaan

1. Jalankan kedua aplikasi dan ambil screenshot kondisi sebelum serta sesudah tombol diklik!

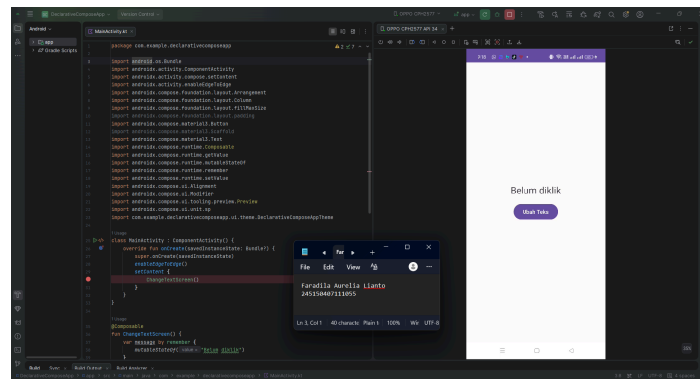
- ImperativeXmlApp (Sebelum)



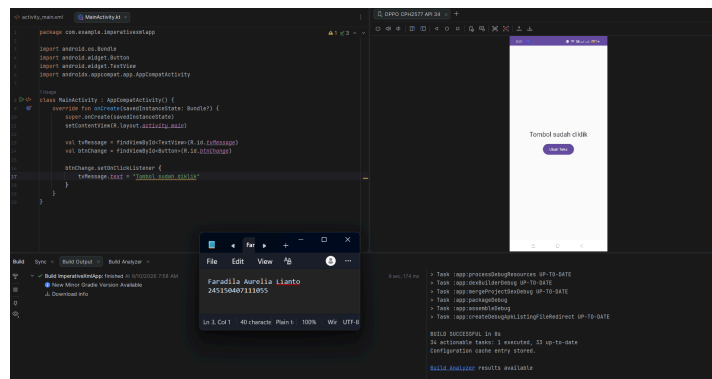
(Sesudah)



- DeclarativeComposeApp (Sebelum)



(Sesudah)



2. Pada project XML/View, objek apa yang diubah secara langsung ketika tombol diklik? Tunjukkan baris kodenya!
tvMessage.text = "Tombol sudah diklik"
3. Pada project Compose, nilai apa yang diubah ketika tombol diklik? Tunjukkan baris kodenya!
message = "Tombol sudah diklik"
4. Mengapa pada Compose tidak terdapat perintah seperti tvMessage.text atau setText()? Karena Compose gak bekerja dengan cara "mencari objek UI lalu mengubah propertinya" (imperatif). Compose bekerja dengan cara "menggambarkan UI berdasarkan state" (deklaratif), komponen Text di Compose bukan objek permanen yang disimpan referensinya kayak TextView, melainkan fungsi yang dipanggil ulang (di-recompose) setiap kali state yang dipakainya berubah. Jadi gak ada objek Text yang "diakses lalu diubah manual", yang ada cuma "state berubah → Compose gambar ulang UI dari awal dengan data terbaru".

5. Jelaskan alur perubahan pada XML/View dengan pola: event → View → perubahan tampilan!

Ketika tombol diklik (event), `setOnClickListener` dijalankan, kode di dalamnya langsung mengambil referensi objek `TextView` (View) lewat `findViewById()`, lalu mengubah properti `.text` milik objek itu secara langsung, sehingga tampilan di layar langsung berubah (perubahan tampilan). Di sini, program yang secara eksplisit memberi perintah "ubah properti ini" ke objek UI tertentu.

6. Jelaskan alur perubahan pada Compose dengan pola: event → state → recomposition → perubahan tampilan

Ketika tombol diklik (event), kode di dalam `onClick` mengubah nilai variabel `message` (state). Karena `message` adalah state yang diamati Compose (dibuat dengan `remember { mutableStateOf(...) }`), perubahan ini otomatis memicu Compose untuk menjalankan ulang fungsi `composable` yang bergantung pada state tersebut (recomposition) — dalam hal ini fungsi `Text(text = message, ...)` dipanggil ulang dengan nilai `message` yang baru, sehingga tampilan di layar ikut berubah (perubahan tampilan). Di sini, program tidak memerintah UI secara langsung — UI otomatis "mengikuti" perubahan data.

7. Apa yang terjadi jika `message` diganti menjadi variabel biasa: `var message = "Belum diklik"`? Jelaskan penyebabnya.

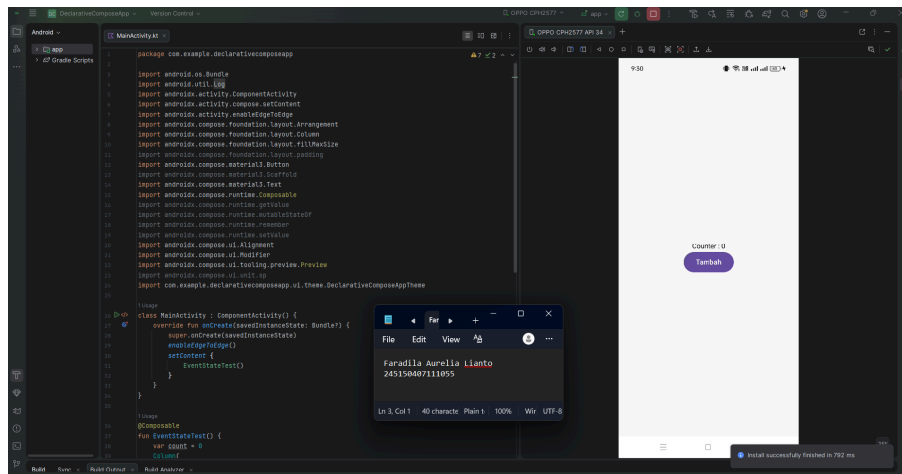
Tombol tetap bisa diklik dan nilai `message` di memori memang berubah, tapi tampilan di layar TIDAK akan berubah, teks akan tetap menampilkan "Belum diklik" walau sudah diklik berkali-kali.

Penyebabnya: variabel biasa (tanpa `mutableStateOf`) bukan state yang "diamati" oleh Compose. Compose cuma tahu harus melakukan recomposition kalau ada perubahan pada objek state yang secara spesifik dibungkus `mutableStateOf()` (dan biasanya dikombinasikan `remember` supaya nilainya tetap tersimpan antar recomposition). Variabel biasa cuma disimpan di memori Kotlin biasa — Compose gak "mendengarkan" perubahannya, jadi UI gak pernah digambar ulang meski nilainya sebenarnya berubah.

8. Tuliskan kesimpulan perbedaan cara berpikir imperatif dan deklaratif menggunakan kalimat Anda sendiri.

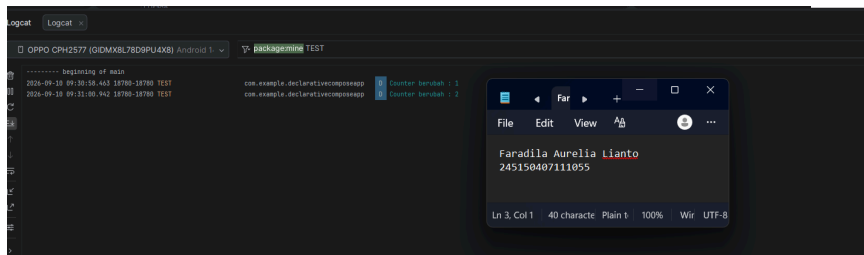
Cara berpikir imperatif itu seperti memberi instruksi langkah demi langkah ke komponen UI tertentu, dan programmer yang bertanggung jawab penuh menentukan objek mana yang harus diupdate dan kapan. Sedangkan cara berpikir deklaratif itu lebih seperti mendeskripsikan, sedangkan programmer cukup mengubah datanya (state), dan framework (Compose) yang otomatis mengurus bagian UI mana yang perlu digambar ulang. Intinya, imperatif itu programmer yang "mengendalikan" UI secara manual, sedangkan deklaratif itu UI yang "mengikuti" perubahan data secara otomatis.

Event Handling State

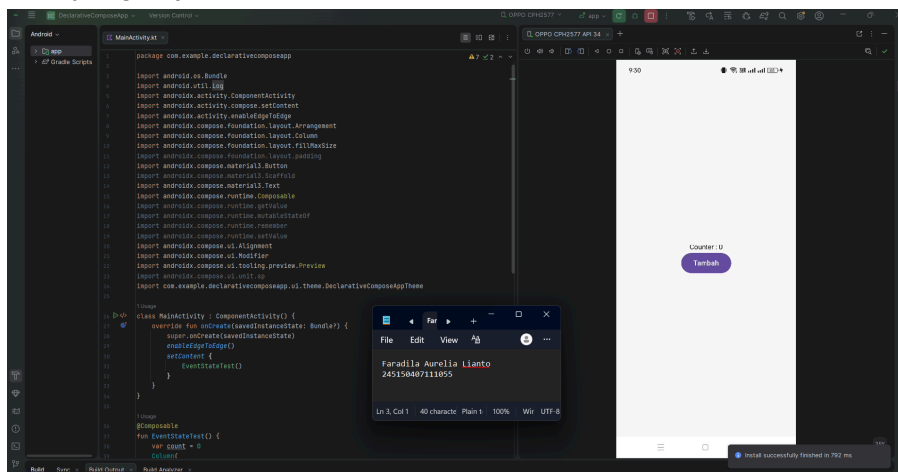


Aktivitas Percobaan

1. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?



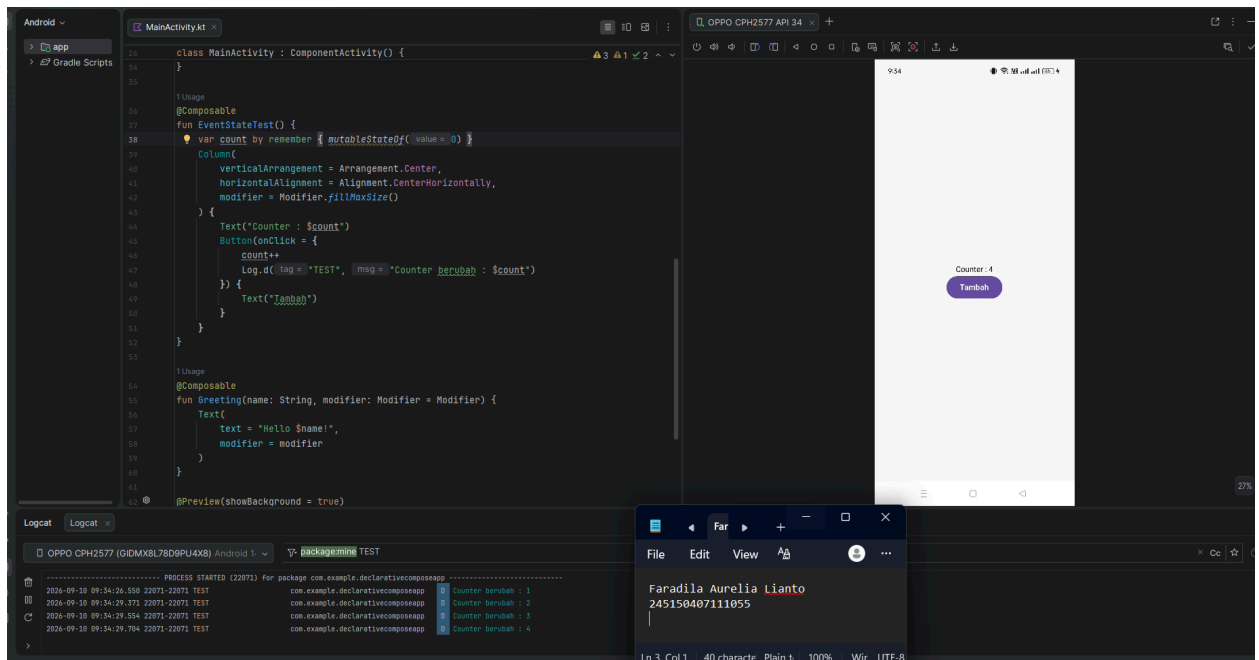
2. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?



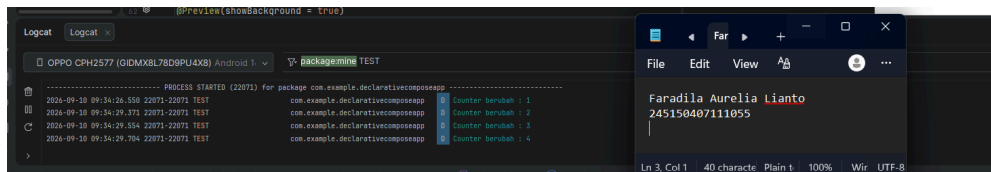
3. Apa yang menyebabkan hal tersebut terjadi?
var count = 0 itu cuma variabel Kotlin biasa, bukan state yang dikenali Compose.
Compose cuma tau harus menggambar ulang (recompose) suatu bagian UI kalau ada state yang diamati (observable state) berubah nilainya. Karena count di sini gak

dibungkus `mutableStateOf()`, jadi walau nilai count sebenarnya nambah terus di memori, `Text("Counter: $count")` gak pernah dipanggil ulang, sehingga tampilannya beku di angka awal (0).

4. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.



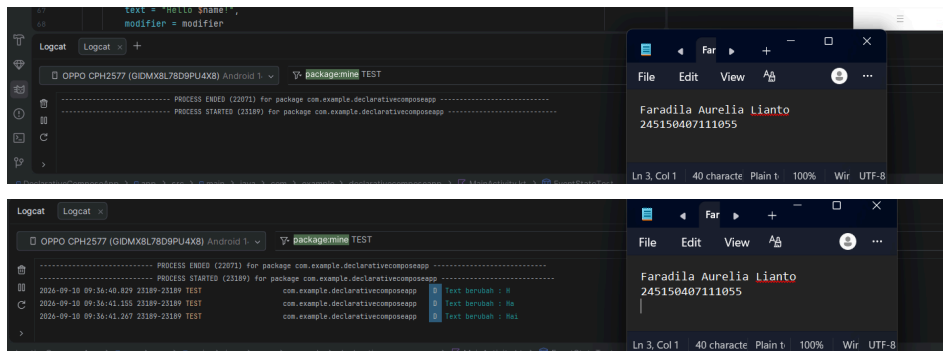
5. Apa yang muncul di LogCat setiap kali tombol "Tambah" diklik?



6. Apa yang terjadi pada tampilan aplikasi setiap kali tombol "Tambah" diklik?
Muncul log baru setiap klik, dengan tag TEST, isinya bertambah sesuai jumlah klik
7. Apa yang menyebabkan hal tersebut terjadi dan berbeda dengan percobaan sebelumnya?
Tampilan berubah. Teks "Counter : 0" langsung ter-update jadi "Counter : 1", "Counter : 2", dst sesuai jumlah klik, secara real-time terlihat di layar.
8. Apa yang dimaksud recomposition pada Compose?
Recomposition adalah proses Compose menjalankan ulang (memanggil ulang) fungsi composable untuk menggambar ulang tampilan UI dengan data terbaru. Recomposition

ini dipicu secara otomatis oleh Compose setiap kali ada state (`mutableStateOf`) yang menjadi dependensi composable tersebut mengalami perubahan nilai, tanpa programmer perlu memerintahkan secara manual objek UI mana yang harus diperbarui.

9. Jalankan program kembali, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik. Kemudian buka LogCat, filter LogCat dengan tulisan TEST. Kemudian klik tombol "Tambah" beberapa kali pada aplikasi, ambil satu screenshot tampilan aplikasi setelah tombol "Tambah" diklik beberapa kali.
10. Apa yang muncul di LogCat setiap kali menginputkan huruf di Text Field?



11. Apa yang terjadi pada tampilan aplikasi setiap kali menginputkan huruf di Text Field? Tampilan pada kotak `OutlinedTextField` langsung ter-update menampilkan huruf yang baru diketik secara real-time, sesuai dengan apa yang diketik pengguna.
12. Apa yang menyebabkan hal tersebut terjadi?

Penyebabnya sama seperti kasus `count` sebelumnya: variabel `text` dideklarasikan sebagai state (`var text by remember { mutableStateOf("") }`). Setiap kali pengguna mengetik, `onValueChange` dipanggil dan mengubah nilai `text` (`text = it`). Karena `text` adalah state yang diamati Compose, perubahan ini memicu recomposition pada `OutlinedTextField` (dan bagian UI lain yang bergantung pada `text`), sehingga tampilan kotak input langsung menunjukkan karakter terbaru yang diketik.

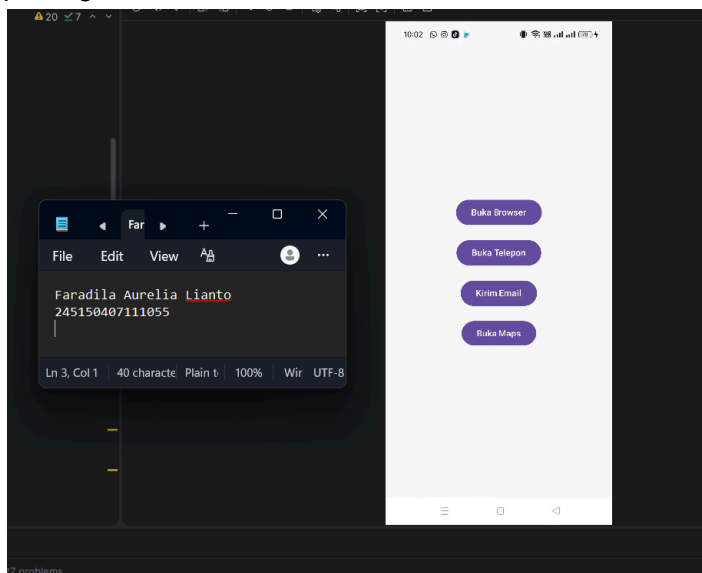
13. Sebutkan state apa saja yang ada pada program yang terakhir dibuat?
 - `Count` : `var count by remember { mutableStateOf(0) }`, menyimpan jumlah klik tombol.
 - `text` : `var text by remember { mutableStateOf("") }`, menyimpan teks yang diketik di `OutlinedTextField`.
14. Sebutkan event apa saja yang ada pada program yang terakhir dibuat?
 - `onClick` — pada Button "Tambah".
 - `onValueChange` — pada `OutlinedTextField`.
15. Kapan event - event tersebut dijalankan?

- onClick dijalankan setiap kali pengguna menekan/mengklik tombol "Tambah". Saat itu terjadi, kode di dalamnya (count++ dan Log.d(...)) langsung dieksekusi sekali per klik.
- onChange dijalankan setiap kali ada perubahan pada teks di dalam OutlinedTextField — baik itu menambah karakter (mengetik), menghapus karakter (backspace), atau paste teks. Setiap perubahan sekecil apapun (per karakter) akan memicu event ini sekali.

Implisit Intent

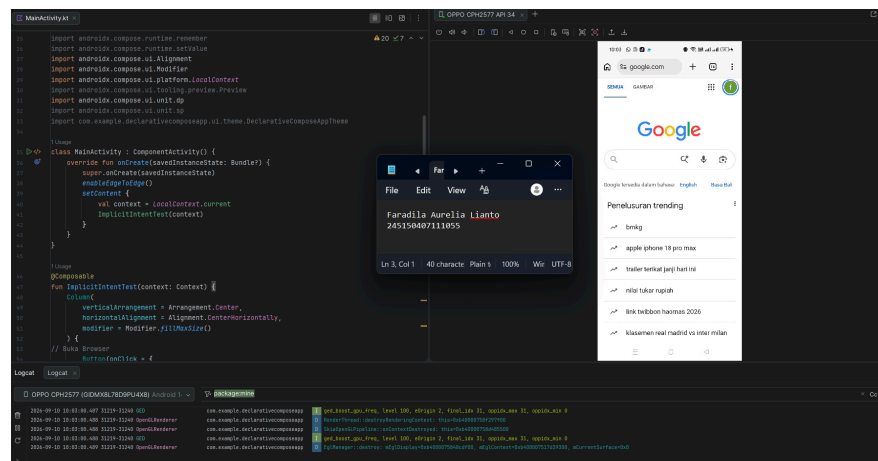
Aktivitas Percobaan

1. Jalankan program, pastikan aplikasi dapat dijalankan dengan baik pada emulator atau perangkat Android fisik.

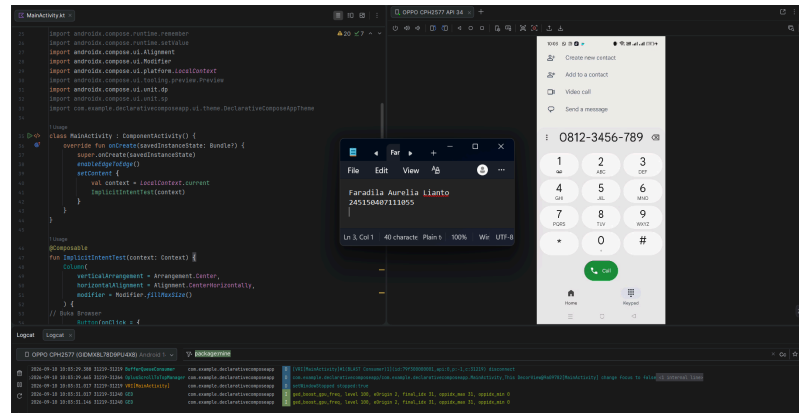


Lakukan pengujian pada setiap tombol:

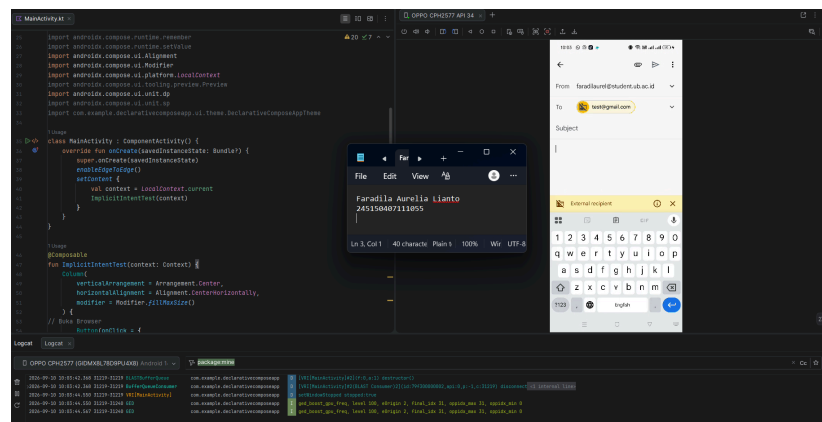
- Klik Buka Browser



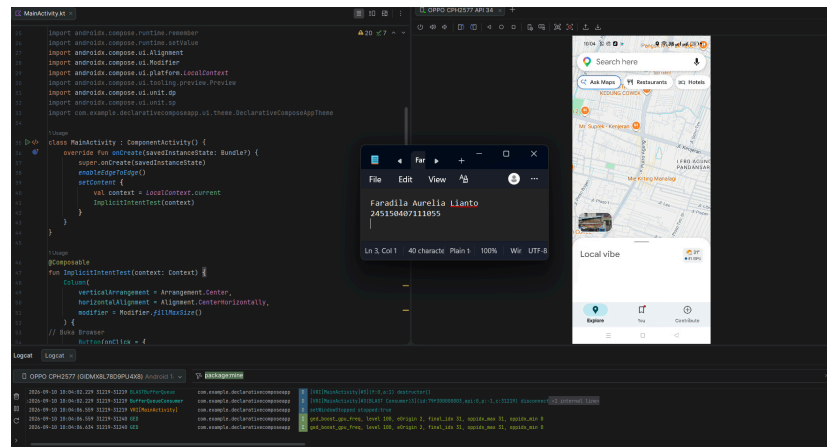
- Klik Buka Telepon



- Klik Kirim Email



- Klik Buka Maps



2. Pada kode yang digunakan, terdapat beberapa action seperti:

- Intent.ACTION_VIEW
- Intent.ACTION_DIAL
- Intent.ACTION_SENDTO

Apa perbedaan fungsi dari masing-masing action intent tersebut?

- Intent.ACTION_VIEW : dipakai untuk menampilkan/melihat data tertentu ke pengguna, sistem akan mencari aplikasi yang bisa menampilkan data tersebut. Kalau data-nya berupa URL web, otomatis akan membuka browser; kalau data-nya koordinat geo, akan membuka aplikasi peta.
- Intent.ACTION_DIAL : dipakai untuk membuka layar panggilan telepon (dialer) dengan nomor yang sudah terisi otomatis, tapi tidak langsung melakukan panggilan. Ini beda dengan ACTION_CALL yang langsung menelepon tanpa konfirmasi (dan butuh permission khusus).
- Intent.ACTION_SENDTO : dipakai untuk mengirim data ke aplikasi komunikasi tertentu berdasarkan skema data-nya (misal mailto: untuk email), sistem akan menampilkan pilihan aplikasi yang bisa menangani pengiriman tersebut (misal aplikasi Gmail, Outlook, dll).

3. Pada kode yang digunakan, terdapat beberapa data seperti:

- data = Uri.parse("mailto:test@gmail.com")
- data = Uri.parse("https://www.google.com")
- data = Uri.parse("geo:-7.250445,112.768845")

Apa fungsi data pada intent ?

data pada Intent berfungsi untuk menentukan informasi/objek spesifik yang menjadi target atau muatan dari aksi (action) yang ingin dilakukan, biasanya dalam bentuk URI (Uniform Resource Identifier). URI ini punya skema tertentu (https://, tel:, mailto:, geo:) yang memberi tahu sistem jenis data apa itu, sehingga sistem bisa menentukan aplikasi mana yang cocok menanganinya.

4. Apa hubungan Action dan Data pada pilihan aplikasi yang dimunculkan oleh sistem Android?

Action dan data bekerja bersama-sama untuk menentukan aplikasi mana yang akan dimunculkan sistem sebagai penanganan Intent tersebut. Action menjelaskan jenis aksi apa yang ingin dilakukan (melihat, memanggil, mengirim, dll), sedangkan data menjelaskan jenis konten/skema apa yang menjadi objek aksi tersebut. Sistem Android akan mencocokkan kombinasi action + skema data ini dengan daftar aplikasi yang sudah mendaftarkan diri (lewat intent-filter di manifest masing-masing) sebagai penanganan kombinasi tersebut